Smart pointers
friend func.
more things
reach constructor of parent class by child class
2nd register
static func.

Date:

LECTURE - 1

Tuesday
21/1/25.

→ POINTERS ← are variable that holds addresses of memory

```
int x;
int * ptr ; // declares a pointer
ptr = &x; // assign adress of variable 'x' to ptr.
```

→ **Constant Pointer:** cannot change the memory address in ptr later in the program.

```
int * const ptr = &x; (name say phelay 'const')
```

- Dereferencing:

```
cout << *ptr ; // this opens up the ptr and let the data in the memory address be accessed.
```

- To change value of variable pointed by the pointer.

```
int x;
int *ptr = &x;
*ptr = 10 // assign '10' to x. OR *ptr += 10 increments by 10
```

→ Pointer to Constant:

```
const int * ptr = x;
x = 12 ✓   (bilkul sahi / main 'const')
*ptr = 12 X
```

For pointer 'ptr', 'x' is constant and x can't be changed using 'ptr' but it can be changed by other means.

- Incrementing Pointer:

```
ptr++; // makes the pointer point the next address allocated.
```

e.g if it is pointing an array, then it will point (store address) of next index of the array.
(works as counter for memory addresses).

Date:

- Pointers passed by value in functions:

```
void function(int * ptr) {
    cout << *ptr ; // outputs the value stored in variable
                   pointed by ptr
}
```

- Pointer passed by Reference:

```
void function ( int * & ptr) {
    static int newValue = 20;
    ptr = &newValue;
}
```

LECTURE-2. Thursday 23/1/25.

→ Dynamic Memory Allocation: → new keyword will always create a space for variable or an array in 'heap' memory which is accessable at run time. THIS LINE MADE A DYNAMIC VARIABLE (on heap)

```
int * ptr = new int;
*ptr = 5;
delete ptr;
```

↓ This will just delete the allocated memory that was created in 'heap' using the new keyword. The pointer 'ptr' will still exist as it was created in stack since 'new' is not used with pointer. (and will always be)

```
ptr = nullptr;
```

↓ Since we've deleted the space allocated in heap memory ∴ we need to assign a null value to pointer so it does not points the memory which does not exist and does not become a dangling pointer.

We cannot directly access heap, so we create pointers in stack, to point the addressess in heap.

Date:

→ Dynamic 1-D Arrays: Only used for user to decide size of Array.

```
cin >> size;
int * Arr = new int [size];
```

* Now this will create a pointer variable named as 'Arr' in stack memory. And, a block memory which looks like array will be created in heap memory (bcs of new) of size the user entered. Therefore the pointer 'Arr' will be pointing the 0th index of that memory block in heap
* We'll be using dynamic arrays with same brackets and same way like we used static arrays.

e.g:

```
for(int i=0; i<size; i++) {
    arr[i] = i ; // Array = {0,1,2,3,4}
}
```

pointer arithmetic

Same as static since arr[i] means *(arr + i) which means the pointer will be incremented by 1 and then dereferenced.

```
delete [] arr; // deletes the memory block in heap
arr = nullptr; // avoids 'arr' pointer to be a dangling pointer.
```

- Dynamic Array Shouldn't be Grown or Shrunked:

If we've a dynamic array of size 3 and we need to add 2 more elements into it, then we won't just add two more indexes in it thinking its dynamic since we don't know what the memory looks like and right after the memory block in heap allocated for size 3 array, there could be any other process's memory which will be disturbed if we simply grow it.

* Same goes for shrinking away.

Solution:

We need to create a new dynamic array of size 5 and then copy paste the elements of size 3 array into the size 5 array. Then we'll delete the size 3 array.

```
int * arr = new int[3];
int * newArr = new int [5];
for (int i=0; i<5; i++) {
    newArr[i] = arr[i]; // copy old array to new array.
}
delete[] Arr; // deletes old array
arr = newArr; // set old array pointer to new array.
// "newArr = nullptr;" this could be done so that we
just use 'arr' pointer to point new array and
'newArr' is not dangling pointer.
```

→ Functions with 1D dynamic Array:

```
void assignValues (int * arr, int size) {
    for (int i=0; i< size; i++) {
        arr[i] = i;
    }
}
int main() {
    int size = 5;
    int * arr = new int [size];
    assignValues (arr, size); // pointers are passed the
}
```

* Pointers are passed by value but they behave like passed by ref.

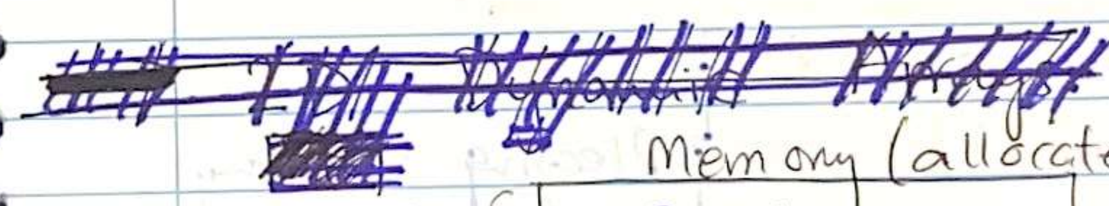
same way as array as they really are arrays here bcs of above line "new int[size]".

```
int * arr = new int[5] {1,2,3-}; } prints address of arr[0].
cout << arr;
char * arr = new char [5] {a,b,c,d,e}; } prints "a,b,c,d,e" as this
cout << arr;                            is special case of C-style string.
```

- Pointer returning function:

```
int * inputArray (int size) {
}
int main() {
    int * arr = inputArray (size);
    for (int i=0; i<size; i++) {
        cout << arr[i];
    }
    delete[]arr;
    arr = nullptr;
}
```

OR we can create a 'new' dynamic array in main and pass it as parameter in func. Then that func. will be 'void' since it won't be assigned to any pointer main like how it was done to 'arr'



- Randomizer Input: (in array)

```
for (int i=0; i<size; i++) {
    arr[i] = (rand() % 7) + 1;
}
```

```
int x = min + (rand() % range)
```

Here values from 1 → 7 will be randomly generated and stored in array.

Hard coding values in 2D Dynamic →

```
int** arr = new int*[3];
for (int i=0; i<3; i++) {
    arr[i] = new int[5]{1,2,3,4,5};
}
```

OUTPUT:
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5

Date:

⇒ 2D Dynamic Array:

this basically creates array of pointers which we extend into column to make it look like 2D array.

```
int rows, cols;
int** arr = new int*[rows];
for (int i=0; i<row; i++) {
    arr[i] = new int[cols];
}
```

Creating a table in heap of "rows x cols".

creates a new array in array[i] index

```
for (int i=0; i<rows; i++) {
    for (int j=0; j<cols; j++) {
        cin >> arr[i][j];
    }
}
```

Taking input (same as static)

((arr+i)+j)

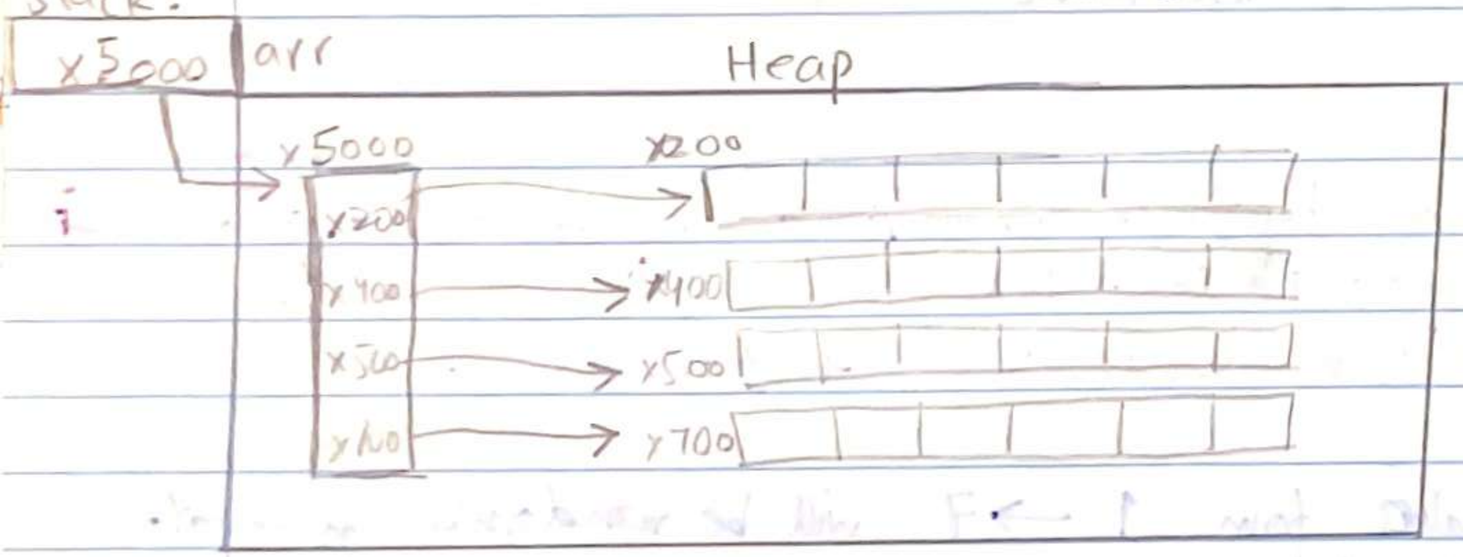
```
for (int i=0; i<rows; i++) {
    delete [] arr[i];
}
delete [] arr;
arr = nullptr;
```

delete array of values (columns)

delete array of pointers (rows)

Releasing memory and dangling pointer

Stack:




```
void AddColumns(int*& arr, int cols) {
    arr = new int[cols]; }
int main() {
    for (int i=0; i<rows; i++) {
        AddColumns(arr[i], i+1);
    }
}
```

int** arr = new int*[rows]; // arr is 2D array.

Date:

LECTURE-3

* Pointers are passed by value in func. but they behave like they're passed by reference since they carry addresses, so the value at the addresses are changed which is basically what happens in passed by ref.

* If pointers are passed by reference like: (int*& ptr) then we can actually update the data in ptr (i.e. the address it stores) but not the data on the address.

```
void ChangePointer(int*& ptr) {
    static int y = 30;
    ptr = &y; // if ptr wasn't passed by reference, then a new
              pointer with y's address will've been created.
}
```

So when func. exits, the address of y doesn't disappear.

⇒ 1D CHARACTER DYNAMIC ARRAY:

```
char* arr = new char[1000];
cout << "Enter name: ";
cin.getline(arr, 1000);
int len = 0;
while (arr[len] != '\0') {
    len++;
}
char* newArr = new char[len+1]; // for null terminator
for (int i=0; i<=len; i++) {
    newArr[i] = Arr[i];
}
cout << "You entered: " << newArr << endl;
delete[] Arr;
```

So null terminator gets copied from Arr → NewArr.

```
int asciiValue = int('a')  // 97 in variable 'asciiValue'
char meow = char('97')  // meow will have letter 'a'.
```

⇒ 2D CHARACTER ARRAY [DYNAMIC]: dynamic 2D ↑ Same as Simple

```
char ** arr = new char * [rows];
for (int i=0; i<rows; i++) {
    arr[i] = new char[cols];
}
```
} Array of pointers

```
for (int i=0; i<rows; i++) {
    for (int j=0; j<cols; j++) {
        cin >> arr[i][j];
    }
}
```
} Input by user which would be complete sentance for which we used "cin". ~~only single characters~~

If we want to convert a 1D char array to 2D; then we'll use "cin.getline"

```
for (int i=0; i<rows; i++) {
    for (int j=0; j<cols; j++) {
        cout << arr[i][j];
    }
    cout << endl;
}
```
} Output in block/table form.

```
for (int i=0; i<rows; i++) {
    delete [] arr[i];
} delete [] arr;
```
} Deallocating heap memory.

Imp ★

```
int arr[3]
```

means: const address of 'arr' + (index * size) i.e: 4 bytes

```
int * ptr = arr[5]; // address of 0th index of 'arr' in 'ptr'.
ptr += 2
```
↳ this makes 'ptr' point 2nd index of arr.

```
cout << ptr[-1]; // prints arr[1] data.
```
2 + (-1 * 4) = ptr/arr 1st index.

★ current address of 'ptr' was made 2 in prev. line

Date

LECTURE-4

Tuesday 18/2/25.

⇒ Classes: To make set of datatypes.

⇒ Access Modifier: ① Public ~~(default)~~
② Private (default).
③ Protected

```
class Student {
private:
    int RollNo;
    string Name;
    float GPA;
public: (for mostly functions)
    int GetRollNo() {
        return RollNo;
    }
    void SetRollNo(int a) {
        RollNo = a;
    }
};
int main() {
    student S1; // 'S1' is object of class 'Student'
    S1.SetRollNo(1234); // "S1.RollNo = 1234" is invalid since "RollNo" is private access modifier.
    cout << S1.GetRollNo();
}
```
} data members.

★ "Getters" and "Setters" are made to access private attributes (variables).
↓ data members

For access of Object (like S1) if it's public

★ After the dot(.) of object (like S1) we can write any variable (like: RollNo) or any func. (like GetRollNo()) THAT IS DECLARED IN CLASS.

Implicit call: called itself and we can't call it ourself (explicit)

always put 'const' in parameter if passing "char array".

⇒ Constructor & Destructors:

Constructor	Destructors
• Special member func. that is called automatically when object is 'created'.	• Special member func. that is called automatically when object is 'out of scope' or 'deleted'.
• Allocates dynamic memory	• De allocates dynamic memory.
• No Return type (not even void)	• No Return type
• Same name as Class.	• Same name as Class with prefix ~
• Use to initialize object's properties.	↳ Good practice to deallocate all dynamic memory there in

→ Default Constructor:

```
Class Car {
Public:
    string Brand;
    Car () { // Defining Default
        Brand = "Unknown";
        cout << "Default Constructor called";
    }
    void Show() { cout << Brand; }
};
int main() {
    Car c1;
    c1.show(); // Default constructor called automatically and handle
    return 0; // as 'c1' goes out of scope here so default destructor
}             called which since is not defined so nothing happend.
```

OUTPUT:

Default Constructor called
Unknown.

goes here ↑

* You can make more than one type of constructors in a class.
* ~~must to make default constructor if using parameterized constructor.~~

→ Parameterized Constructors. (overloaded Constructor).

```
public:
    Car(string b) { // defining parameterized constructor 'car()'.
        brand = b;
    }
int main() {
    Car C1("Toyota"); // sets value of brand to "Toyota" and
    C1.print();       // prints it as "parameterized Constructor"
}                     // is called here when object 'c1' is created
```

Car() { } ← default constructor necessary for parameterized func

* We can get input from user in variables and then pass em as parameter

→ Constructor with Default Arguments.

```
public:
    Car(brand b="Toyota") {
        brand = b;
    }
int main() {
    Car C1; // will print 'Toyota' since 'brand' set to 'b' which is set↑ to 'Toyota'.
    Car C2("BMW"); // will print 'BMW' as the parameter
}                  // in func. of constructor overrides default argument.
```

→ Destructor:

```
public:
    string brand;
    Car(string b) { // default Constructor
        brand = b;
        cout << "Constructor for " << brand;
    }
    ~Car() { // defining default destructor
        cout << "destructor for " << brand;
    }
```

```
int main() {
    Car C1("BMW"); // default constructor called
    cout << C1.brand; // prints "BMW".
    return 0; // default destructor called
}
```

OUTPUT:
Constructor for BMW
Destructor for BMW

→ Scope Resolution Operator ::

- It is used to access members or functions of a "class" or "globle scope"
- We can declare func. inside class and define it later in program or in a header file (.h) to make our class body look cleaner. This is done by :: to tell that specific func. we are referring to and are defining belongs to a specific class.

```
class Car {
public:
    Car();
    ~Car();
    void Print();
};
Car::Car() { cout << "Constructor"; }
Car::~Car() { cout << "Destructor"; }
Void Car::Print() { cout << "Meow"; }
int main() {
    Car C1;
    C1.print();
}
```

Signatures/ declaration of func.

Defining function outside body of Class.

OUTPUT:
Constructor
Meow
Destructor

Date:

★ Scope Resolution operator can also be used to refer to "global variable" as it over rides local variable.

```
int value = 50;
int main () {
    int value = 20;
    cout << value;      // output 20
    cout << :: value;   // output 50
}
```

Pointers goes "out of scope" (or dies) Object gets deleted.

⟹ SMART POINTERS: To "automate" the process of writing 'new' (to allocate dynamic memory) and 'delete' (to deallocate dynamic memory).

- When smart pointer will be created, the memory in heap will be allocated and then based on which type of smart pointer we are using, that memory will be at some point be automatically deallocated

- #include <memory> require for smart pointers.

⟹ Types:

→ Unique Pointers: Only one pointer can hold address of a memory.

```
int main() {
    int a = 25; // 'a' is on stack
① unique_ptr<int> Ptr1 = make_unique<int>(a);   // one of the one-lation to make a unique pointer
② unique_ptr<int> Ptr2 = Ptr1;
③ unique_ptr<int> Ptr2 = move(Ptr1);
④ cout << *Ptr2;
⑤ cout << *Ptr1;
⑥ }
```

Constructor, new, scope in are done at one instance
Destructor, delete, scope out are done at one instance.

★ M.L = memory location

Date:

① Creates a unique pointer named "Ptr1" and it is now pointing to a memory location on HEAP which has stored the value of 'a' in it. (Basically a variable on-heap).
('new' keyword replaced)

② INVALID. Since two pointers can not point same memory location bcs if Ptr1 gets deleted; Ptr2 will've nothing to point.

③ move() func. gives the address of M.L to 'Ptr2' and 'Ptr1' is automatically pointed to 'nullptr'.
(Ptr1=NULL replaced)

④ Output: 25

⑤ INVALID. Since Ptr1 does not exist now.

⑥ Delete called for deallocation

```
public:
    MyClass(); // Constructor
    ~MyClass(); // destructor
int main() {
    {                                   // extra scope
// destructor invoked with 'new'       // unique_ptr<MyClass> Ptr1 = make_unique<MyClass>();
    }                                   // destructor invoked along with delete.
}
```

unique pointer of type "MyClass".

★ If we've smart pointers of a single datatype like 'int', then we can assign values in ( ) as 'a' was assigned on previous page. In this case we can also do *ptr1 to get the value at that address. But in case of smart pointers with 'class' type we can't dereference.

→ **Shared Pointer**: Multiple pointers can have address of same memory. strong

- Have a counter named as "Reference" which tells the no. of shared pointers holding any specific address. That reference only increments if any other "shared pointer" points that location; NOT when a "weak pointer" points that data.
- If at least one shared pointer is pointing an object, it can't die. When the last shared pointer holding object's location goes out of scope, the object is deleted from heap.

```
public:
    MyClass();
    ~MyClass();
int main() {
    { // Starting scope of Ptr1
// Constructor of Ptr1 invoked → shared_ptr<MyClass> Ptr1 = make_shared<MyClass>();
        cout << Ptr1.use_count(); // output: 1 (i.e Ptr1).
        { // Starting scope of Ptr2
// Constructor of Ptr2  shared_ptr<MyClass> Ptr2 = Ptr1;
            cout << Ptr1.use_count(); // output: 2 (i.e Ptr1 and Ptr2)
        } // destructor of Ptr2 OR Ptr2 ending scope.
        cout << Ptr1.use_count(); // output: 1 (since Ptr2 died)
    } // destructor of Ptr1 OR Ptr1 ending scope.
}
```

function to count strong reference of shared pointers.

→ **Weak Pointer**: They holds a shared pointer but does not acctvally holds the memory address of object (i.e: cannot change the value of memory address by "dereferencing" until a special "lock()." func. used)

- Ptr.lock() converts a weak pointer into shared pointer ONLY if the object is still alive (means that any one of the shared pointer holding an object has not gone out of scope and is still alive).

If object is deleted, then lock() returns **null**.

```
int main() {
    weak_ptr<int> wPtr;
    { // scope of shared pointer 'sPtr'
// 'new' memory on heap created  shared_ptr<int> sPtr = shared_ptr<int>(25);
        wPtr = sPtr;
    } // memory on heap (object) deleted since shared pointer goes out of scope.
① if (auto newPtr = wPtr.lock()) {
② cout << *newPtr;
    } else {
③ cout << "Object No longer available";
    }
} // wPtr goes out of scope.
```

① Statement can also be written as: shared_ptr<int> newPtr = wPtr.lock(); ← converted weak→shared pointer.
if ([redacted] newPtr)

② '25' wld've been printed if the IF-ELSE was inside the scope of 'sPtr' (i.e: the bracket above it).

③ Since IF statement failed (as wPtr.lock() returned false bcs of object being deleted bcs of sPtr out of scope), so final output will be: "object no longer available".

LECTURE - 6

Tuesday. 4/3/25

⟹ Parameterized Constructor with default Arguments:

```
public:
    Rectangle ();
};
Rectangle::Rectangle(int l=0, int w=0) {
    cout << "Constructor with default arguments";
}
int main() {
    Rectangle r1;
    Rectangle r2 (5,7);
}
```

* We made a constructor with default arguments here which will be used as default constructor with default values (0,0) in case of 'r1' and as a parameterized function in case of 'r2'.
* But the benefit is that we don't have to make "default" and "parameterized" constructors seperately.

---

r1 = r2;
This is valid statement and length of r1 and with of r1 will be made 5 and 7 respectively from (0,0) previously.

---

cout << r1;
↳ we can overload this operator '<<' by redefing it in our class which will basically print all data members of r1.

LECTURE - 7

Tuesday 11/3/25.

→ Create pointer of any class type on stack.

```
int main () {
    Rectangle * rectPtr; // rectPtr can only point Rectangle manag.
    Rectangle r1 (3,4);
    rectPtr = &r1; // rectPtr stores address of r1.
    rectPtr -> Print () // (*rectPtr).Print();
    cout << sizeof (rectPtr)
}
```

→ Create pointer of any class type on heap:

```
Rectangle * rectPtr = new Rectangle (100, 200);
```

```
{ Rectangle * rect1; // Constructor
  Rectangle * rect2 = new Rectangle;
  delete rect2;
}
```

→ Constructor and Destructor of static and Heap objects:

```
int main () {
    Rectangle * rect1; // Constructor of rect1
    Rectangle * rect2 = new Rectangle; // Constru. of rect2
    if (rect2) { // OR (rect2 != 0) means it exist. except saves from
        delete rect2; // destructor of rect2
    }
} // destructor of rect1.
```

inside destructor

* The constr./destr. of static & heap is for int main() since there is no thing like scope inside class.

⟹ Constructor / Destructor of Heap objects (variables):

```
int main () {
    Rectangle * rect1;  // no constructor since only pointer ↑ with no address pointed
    Rectangle r1;  // Constructor of r1
    rect1 = &r1;  // just assignment so no constructor
    Rectangle rect2 = new Rectangle ();
    delete rect2;  // destructor of rect2.
} // destructor of r1
```

constructor of 'rect2' called since space allocated on heap.

- Two things could be created on heap; ① objects and ② variables/arrays inside class.
- Objects need to be deleted in main func. (like above) to call its destructor. Even if the program leaves the scope of int main(), destructor of heap objects won't be called until "delete" keyword. delete rect2;
- The variables/arrays created inside class on heap needs to deallocate their memory in destructor along with pointer pointing to nullptr.

⟹ Constructor & Destructor of Heap Array as object:

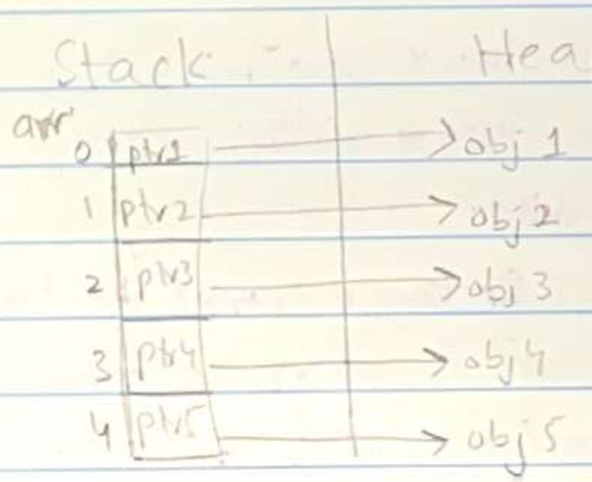
```
int main () {
// Constructor arr1 //  Rectangle * arr1 = new Rectangle [5];
// Constructor arr2 //  Rectangle arr2 [5];
* Both Constructors are called 5 times
    for (int i=0; i<5; i++) {
        arr1/2[i].setValues (i+1, i+2);   // or use parametrised constr. (4 pages ahead)
    }
    delete [] arr1;  // destructor arr1 called 5 times in reverse order.
    arr1 = NULL;
} // destructor for arr2
```

```
for (int i=0; i<5; i++) {
    (*arr[i]).display();  // since "arr[i]" is a pointer to an object instead of object itself
}
```

→ Array pointing to objects ✗

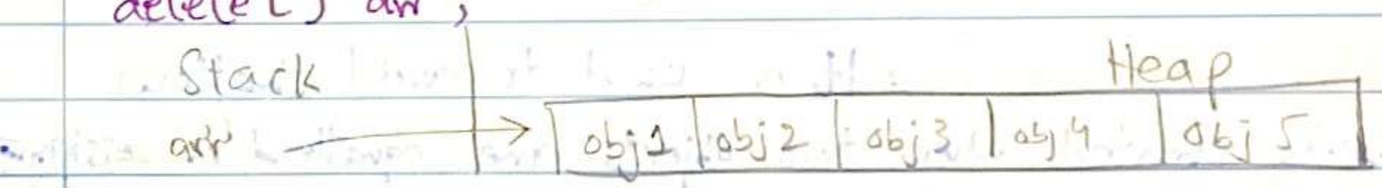
```
Rectangle * arr[5]
for (int i=0; i<5; i++) {
    arr[i] = new Rectangle (2,3);
}
for (int i=0; i<5; i++) {
    delete arr[i];
}
```

delete each object



→ Array of objects itself ✓

```
Rectangle * arr = new Rectangle [5];
delete [] arr;
```



* This happens bcs compiler do work differently for both while reading line wise.

LECTURE -8

~~Sat~~ Thursday 13/3/25.

⟹ Copy Constructor: It is used so we can copy all the attributes of one object to another object completely

* If we won't make a copy constructor, there wld be a shallow copy of object created causing problems.

```
Car c1;
c2 = c1;
c2.initialise();
Car c2 = c1
```

All three uses copy constructor.

In both copy constructor & operator overloading, core thing is jis object pe call hota hai uskay attributes main jo object as parameter pass hota hai uskay attributes ③ dedye. after ② creating new memory if the attributes are pointers (an extra step of ① deletion first is there in equals to overload.)

[Syntax] :

Rest code for dynamic char array

```
private:
    int * length;
    int * width;
    char * title
public:
    Rectangle (const Rectangle & obj) {
        length = new int (* obj.length);
        width = new int (* obj.width);
    }
```

len = strlen(obj.title)
title = new char [len + 1];
for (int i = 0; i <= len; i++)
title [i] = obj.title[i]

OR
length = new int;
* length = *(obj.length)
width = new int;
*(this→width) = *(obj.width);

this will be length of r2 being assigned with length of r1

```
int main () {
    Rectangle r2 = r1;
}
```

* r2 invoked copy constructor & * r1 is passed by ref. in it

* Copy constructor only invoked when r2 is created as object and assigned with r1 IN A SINGLE LINE.
* Copy constructor also invoked when object is passed by value.

⟹ Operator Overloading : It is used to avoid shallow copy being created when two objects are equated/ one object is assigned to another object.

'=' only needs to be overloaded if the class have private members on heap i.e. pointers like:
private:
int * speed
so a shallow copy on heap isn't created

Use Cases:
c2 = c1;
c2 = initialise (); → return type is a class

* If we won't overload operator, then simple = will create a shallow copy; if we'll, then a deep copy will be formed.
No need for OO in static variables since object gets created on different memory locations on 'stack', so shallow copy problem won't exist.

[Syntax:]

return type: class ← Rectangle & operator = (const Rectangle & obj) {
(function name (keyword)) (passing object by ref)

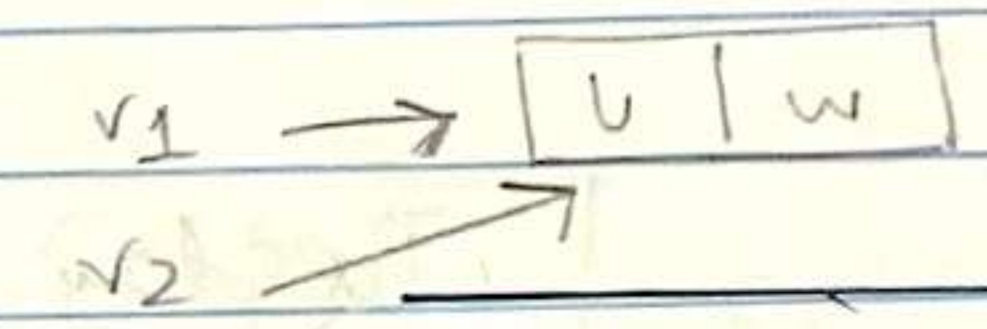
```
    if (this != &obj) { // self assignment check i.e r1=r1.
        delete speed;
        speed = new int;
        *speed = * (obj.speed);
    }
    return * this; // returns the object who called it
}
```

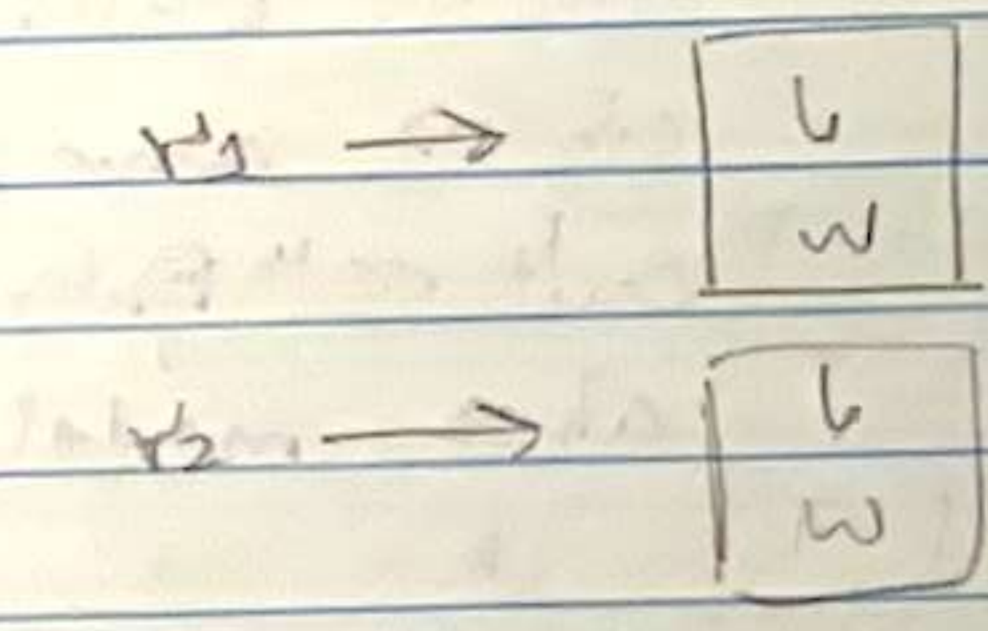
for cascading effect : a=b=c

which contains all the values of private members.
e.g: r2 = r1. r2 called this func. and r1 was passed by ref.
② but members are on heap/pointer ① since obj on stack

When "r2 = r1" OR "Rect r2 = r1" is done:

Shallow Copy:	Deep Copy:
Creates a new object r2 but instead of copying actual data, it copies the references (pointers) to the original data (in r1). ∴ If r2 gets out of scope, it will delete the actual data as well since it was pointing it and r1 would be dangling pointer.	Creates a new object that copies actual data from r1 and stores in r2.





⟹ Passing Object as parameter.

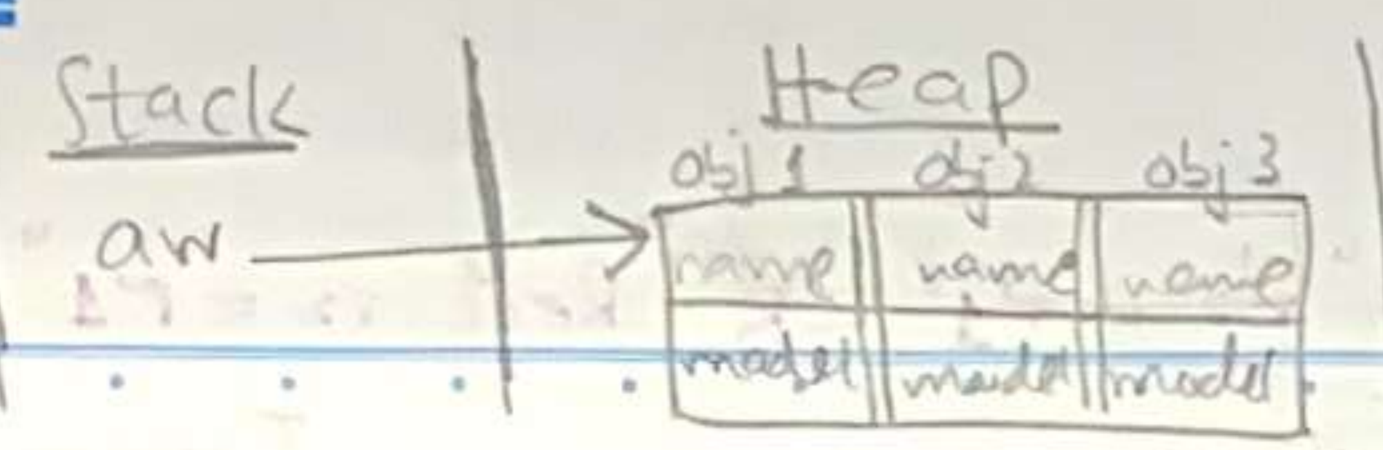
```
Bool Rectangle :: Isequal (Rectangle obj) {
    if (* length == * (obj.length)) // (r1.length == r2.length)
        return true;
}

int main () {
    r1.Isequal (r2);
}
```

If overloaded func. defined outside class.

```
Rectangle & Rectangle :: operator = (const rectangle & obj) {
```

[NOTE:]
- If an object is passed by reference in a func. no constructor is called since the func. is referring to original object and no new object created.
- If an object is passed by value, copy constructor is called since the func. needs new copy of object to work with. So while creating that copy without a shallow copy, copy constructor needed.



⟹ Parameterized Constructor in 1D Array:

```
Car * arr = new Car [size];
for (int i=0; i< size; i++) {
    cout << "Enter name";
    cin >> name;
    cout << "Enter Model";
    cin >> model;

    arr [i] = Car (name, model);
}
```

LECTURE - 9

Tuesday 18/3/25.

→ Overload '+' to add complex no: (c1 + c2)

(class name)

c1 + c2 — * 2 objects added

```
Complex& Complex :: operator + (const Complex & obj) {
    Complex result (real + obj.real , imaginary + obj.imaginary);
    return result;
}
```

(could be done without this)

c1 + 3 — * object and int added.

```
Complex& Complex :: operator + (const int &a) {
    Complex result (real + a , imaginary);
    return result;
}
```

⟹ ! NOT overloading:

```
bool Complex :: operator ! () { 
    return (real == 0 && imaginary == 0);
    ↳ //OR return (!real && !imaginary);
}
```

(no parameter. jisnay call kia wo he use hoga.)

(these are default ! NOT)

(object on left side of == will be calling object & on RHS wld be parameter)

⟹ == operator overload:

```
bool Complex :: operator == (const Complex & obj) {
    return (real == obj.real && imaginary = obj.imaginary);
}
```

(real → c1, obj.real → c2)

```
int main () {
    if (c1 == c2) // overloaded == called
        cout << "c1 equal to c2";
```

LECTURE - 10

Thursday 20/3/25.

★ We can also change (return) datatype in operator overloading. like for '>=', we can return a complex no.

⟹ Friend Func: For overloading '>>' & '>>'

```
class Complex {
    global { friend ostream & operator << (ostream & out, const Complex & obj2);
    class func. { friend istream & operator >> (istream &, complex &);
private:
public:
```

(return type; keyword for 'cout'; object to output)

Syntax of cout << overload:

```
// globally defined
ostream & operator << (ostream & out, const Complex &obj) {
    out << obj.real;
    if (obj.imaginary >= 0) {
        out << "+";
    }
    out << obj.imaginary << "i";
    return out; // for cascading: { cout << c1 << endl;
                                  { cout << c1 << c2;
}
```

(outputting more than one object)

Syntax of cin >> overload.

Date: object to take input in

```
istream & operator >> (istream & in, Complex & obj) {
    cout << "Input Real value: ";
    in >> obj.real;
    cout << "Input Imaginary value: ";
    in >> obj.imaginary;
    return in; // for cascading: in >> c1 >> c2;
}
```

- 'Friend' func. ↑ are non member func. used inside class so we can access private members of class ∴ we gave signature in beginning ↑ (can be inside/outside private/public).
- If we won't give signatures & simply define the func. in global body (without scope resolution operator) like we did on previous page, then we won't be able to access privat members directly like: obj.real;. we wld have to make getters for that like: obj.GetReal();

Q: When else to use friend func.?

* In cout << c1; the calling func. is 'cout' object (which is of 'iostream' class). Therefore when the calling func. is an object of any other ↑ different class, we always have to use friend func. (e.g.: cout & c1 are not of same class)
- Similarly if it is 2 + c1, we wld still make a friend func. for it since the calling func. is '2' which is object of 'int' class, where c1 is object of 'Complex' class.
* But for simple c1 + 2, we'll simply use '+' overload like done previously since calling func. is 'c1' and it is of same class i.e: 'Complex' in which we overloaded the '+' operator.

at end of oop section

⟹ Overloading pre/post increment (++):

- Pre increment (++c1):

```
Complex & Complex :: operator ++ () {
    real++;
    return * this
}
```

- Post Increment: (c1++) → to differentiate b/w post & pre increment

```
Complex & Complex :: operator ++ (int) {
    Complex temp = * this; // OR Complex temp (*this);
    real++;
    return temp;
}
```

* Post increment will return original object ↑ first which was passed and then increment 2+3i → 3+4i.

⟹ Diff b/w normal class func. & friend func:

- Friend func. can be given signature or declared anywhere in class (whether private, public or external area)
- Friend func. is called with object in argument like: print(Box) whereas normal func. is like Box.print().
- 'Friend' keyword puts that func. out of scope of the class. but it can still access private & proteded members of the class where we gave its signature.
- Friend func. isn't the part of class & its differentiator form normal func. is just that, & its also used for operator overload of >>, << (istream & ostream).

* Inside a friend func definition, we cannot use the name of private member like 'real'. we need to do 'obj.real' where 'obj' is the parameter → see e.g of in/out on prev. page.

Date:

⇒ Friend func. for 'cin' into a private member, character array on heap, of a class.

```
Class MyString
pirate:
    char * s;
    int length
public:
    friend istream & operator >> (istream & in , MyString & obj) {
        char temp [1000]; // static char array for fetching data
                          :
        in.getline (temp, 1000); // to read all char in a line
// ensures deep copy   obj = MyString (temp);   at once - CANNOT DO
        return in;                                "in >> *(obj.s)"
    }
}
```

int main () { MyString str1; }

IMP: ★ We can't directly do "cin >> str1" since 'cin' is object of 'istream' class and 'str1' is object of 'MyString' class. Since both are diff. we need to overload operator for input.

★ An alternate would be str1.setString ("Cat") but for that we will have to create setters for every string member used in class.

```
class Meow {
    int * a;
public:
    Meow (int value) : a (new int) {
        *a = value;
    }
}
```

Member Initialiser for pointer variables / Array.

Date:

LECTURE - 11

Thursday 27/3/25.

~~ASSOCIATION OF CLASSES~~

⇒ Member Initialisation List: is a way to initialize class member variables before the constructor body executes.

★ Used because we can not assign "constant" and "reference variable" (int &y) in constructor because till the constructor body is created, the variables are already existing in private, and can no longer be changed since they're constant.

★ More efficient than using a "constructor" for initialisation.

```
class Meow {
    int a, b;
public:
// parameterised constructor
    Meow (int length, int width) : a(length), b(width) {
        cout << "length = " << a;
        cout << "width = " << b;
    }
}
int main () {
    Meow m1 (10, 20);
}
```

initialises a = length and b = width

```
Meow () : a(10) , b(20) {...} // Default constructor with initialization
int main () {
    Meow m1 ; // no arguments
}
```

LINKING CLASSES:

Association
- Aggregation
  - Composition

Mostly Short Q/S

"uses - a" (1→many, many→1, many→many)

⟹ ① ASSOCIATION: A way of connecting classes in a way that both the classes are indepadent to eachother. Means if the object of one class is destroyed, it won't affect the object of other class.

- This is done with a pointer object which makes one class know about other's object but doesnot owns it.

```
class Bulb {
    private:
        char color [10];
        ⋮
}

class Switch {
    private:

    public:
    void operate (Bulb * b){...}
}
```

```
int main () {
    Bulb b1;            | Bulb* b;
    switch S1;          | Switch*S;
    S1.operate (& b1);  | S->operate(b);
}
```

★ We linked the classes independently by making a pointer object of 'bulb' class inside of 'Switch' class. (operate func.)

⟹ ③ Composition: (Strong ownership — "part-of")

Here there is an ownership of one object to another. This wld be one way relation where one class is bigger than the other. If A class owns an object of B class; and object of A gets destroy; then the object of B will automatically be destroyed. But if object of B gets destroyed, it won't affect object of A.

★ Here the object of the other class in own class is not created with pointer due to which an owned relationship is formed.

⟹ DIFF. B/W ASSOCIATION & AGGREGATION.

- In Association, the object of other class can only be passed in any func: whereas in Aggregation, object can be passed in both constructor and func. (Even this is also association)
- In aggregation, the pointer to object of other class is made inside private whereas in simple association, it is not.

aggregation:
```
class switch
    private:
        Bulb * b;
    public:
```

one child can only have one parent in aggregation

```
B   class Engine {
        ⋮
    }

A   class Car {
    private:
        Engine e1; // 'Car' directly owns the 'e1' object of 'engine'
                   // without the pointer, so if 'c1' will
        ⋮          // be destroyed, 'e1' will also be destroyed.
    }

    int main() {
①       Car C1;
②   }
```

→ Creation / Destruction cycle:

① e1 created
   c1 created

② c1 destroyed
   e1 destroyed.

⟹ ② AGGREGATION: Special type of Association BUT (Has - a) relation.

The classes are linked but does not control lifetimes of e/o objects.

WHOLE (Library), PART (Books): relation

Child cannot belong to any other parent

```
class Bulb {
    ⋮
};

class Switch {
    private:
        Bulb * b1;
    public:
        Switch (Bulb * bulb) {
            b1 = bulb;
        }
```

```
int main() {
    Bulb b1;
    Switch S1 (& b1);
    S1.Turnon();
}
```

just the diff. b/w this and association is that here the bulb's object is passed in constructor of switch; but in association, object 'b1' is passed in normal func. of switch class

Base Class → Parent Class
derived class → child class

LECTURE - 12.

Tuesday
15/4/25

⟹ INHERITECE : IS - A.
For child class to inherit all the attributes and functions of parent class.
* Student is - a Person.

Types: ★ Child class shld always has a unique member other than parent member.
~~Types:~~
~~① Public: only access public members.~~
① Public: makes all the inhaited members public for the child class. (e.g: no need for getter/setters for inherited members) - Like private variables in parent class will become public in child.
② Protected: Makes

→ Types:
① Public: The public members of parent class will remain public in child class, private will be private and won't be able to get accessed directly wout getters/setters (which are public func.).
② Protected: Parent's protected members and public members get protected mamber of derived/child class
③ Private: All; public, private & protected gets private.
To use private, we'll make a public class e.g: "Void Name" that will have getName() func. inside it which will get the private name from parent class.

Inheritance type	Parent public becomes	Parent protected becomes	Parent private becomes
public	public	protected	inaccessible
protected	protected.	protected	inaccessible
private	private	private	inaccessible

⟹ Examples of Types of Inheritance.

Parent Class (Same for All):

```
#include iostream
using namespace std;
class Base {
private :
    int value;
public :
    void setValue (int v) {
        value = v;
    }
    void . getValue () {
        return value;
    }
}
```

① Public:

```
class Meow : public Base {
public:
    void display() {
        cout << "Meow";
    }
}
```

★ In public, we can access all except parent's and child's private.

```
int main() {
    Meow obj 1;
    obj. SetValue(20);
    cout<<obj1. GetValue;
    obj1. display();
```

Date:

② Protected:

```
class Meow : protected Base {
public:
    void display() {
        setvalue(20);
    }
};
int main() {
    Meow obj;
    obj.display(); // obj.setValue(20) } wrong since they're
                   obj.getvalue()     } now protected members
}                                       of parent.
```

* Here setvalue is inside a public func. of child class i.e. display().

③ Private:

```
class Meow: private Base {
public:
    void display() {
        setvalue(20);
        cout<<getvalue();
    }
};
```

```
int main() {
    meow obj;
    obj.display();
    // obj.setvalue(30) is
    wrong since setvalue
    is private now
```

made a new public func. and have the public func. 'setvalue' of base class inside it since that public func. is private now and cannot be accessed w/out the public func. 'display'

IMP: To access the private variables of Base class in child classes; don't make them private; Make them "protected" in Base class

Date:

⇒ Syntax of Inheritance & Constructor/Destructor Invoke.

```
class Animal {
public:
    Animal() {
        cout<<"Animal Constructor";
    }
    ~Animal() {
        cout<<"Animal Destructor";
    }
    void eat() { ... }
};
```

```
class Dog: public Animal {
public:
    Dog() {
        cout<<"Dog Constructor";
    }
    ~Dog() {
        cout<<"Dog Destructor";
    }
    void bark() { ... }
};
```

```
int main() {
    Dog d1;
    d1.eat();
    d1.bark();
}
```

Output:
Animal Constructor
Dog Constructor
Dog Destructor
Animal Destructor.

* First the constructor of Parent class, then child classes.

LECTURE 13

Tuesday 8/4/25

⇒ Reference variable:

```
int a;
int& meow=a;
meow = 10; // assign 10 to 'a'
cout<< a; // prints 10.
```

* Used in 'By REF' value inside func. parameter:

```
void change (int & x) {
        x= 100; }
```

NOTE:
- Once the ref variable 'meow' is assigned with 'a', it cannot be assigned with any other variable till end.
- Reference variable should be declared and assigned in same line.

⇒ R/L value:

a = 5;

left value: variable.
right value: temp value w/out name. (can't call it again)

```
int & ref = a; // ref is alias for 'a' (value)
int && meow = 5; // meow is alias for '5' (r value).
        meow is 'R value reference'.
```

* They're used for move semantics.
  - only if memory inside class is ~~transferred~~ on heap.

⇒ Move Semantics: Stealing resources (like memory) from one object to another instead of copying which saves resources.

v2 = obj;

- We can either use copy constructor ↑ to copy all data in v2 from obj, but that would be impractical & expensive in case of large data.
- Therefore we use "move constructor" and "move assignment operator" to transfer the ownership of 'obj' to 'v2'.

Meow

- We'll take all the pointers pointing memory locations of "obj's" members, and will transfer the ownership of those pointers to "v2" so it can access the members now.
- But "obj" will be empty (but still accessible) now. we should'nt read data from here onwards.

⇒ Move Constructor:

```
class MyClass {
    int * data;
public:
    MyClass (MyClass && obj) noexcept {
        data = obj.data;
        obj.data = nullptr;
    }
}
```

noexcept → for vectors etc.

took ownership from "meow" and transfered to 'v1' when "v1 = meow"

* Used when "Rectangle v2 = move (v1);

⇒ Move Assignment Operator:

```
MyClass & operator = (MyClass && obj) noexcept {
    if (this != &obj) {
        delete data;
        data = obj.data;
        obj.data = nullptr;
    }
    return * this;
}
```

* diff. btw this & simple '=' overload is that this just transfers the pointer whereas the normal '=' overloaded copy's all data.

* Used when:

C2 = move (C1)

(.)
use →
variable, then no *(_.→_)
pointer, then do *(_.→_)
e
value).
~~instead of pointer variable,~~
~~instead of *(arr b1. arr~~
p e.g. "char * title", we
. title[i]". No need for *
need for pointer variable.

C1. operator + (3) and
func. Only '+' operator needs

e both in one class (i.e. complex)
friend.

operator+(C1) and since operator '+'
calling func's (i.e: '3') class
need friend func.

~~⇒ This : 'this' is basically 'a pointer to object'.~~

```
class Bulb {
  private:
      int v1;
      int *v2;
  public:
      void state () {...}
      void GetV1 () {...}
      Void Ge
}
int main () {
```

~~bulb.v1~~
~~bulb.state~~

This : this is basically 'a pointer to object'. Only used inside class

```
class Bulb {
public:
      int value;
      int * ptr;
      void state () {
      int GetPtr () {
          return * ptr
}
```

```
int main() {
    B... b1;
    ... b2;
```

'dot' if we're object
'→' if we're pointer to object.

```
b1.value;
b1.state();
b2 → value;
b2 → state();
this → value;
this → state();
*(this → ptr);
this → ptr;    for access...
*(b1.ptr); // b1.GetPtr()...
b1.ptr;    for access...
*(b2 → ptr);
```

(*b2).value
(*this).value

th... ...ue ...ways wro... 'this' is
t... ...ate ... pointer ... be used
...th a ...t dereferencing.

...vai...
...: s...
...e:
...() ... func.
... returning func.
... func.
... pointer func.

Simple: For program with 3 class and global func. Make:

- 3 header for class signature
- 3 cpp for class definition
- 1 header for global func. sig.
- 1 cpp for global func. def.
- 1 main.cpp.

⟹ Classes with Header & .Cpp files

main.cpp

```
#include <iostream>          using namespace
#include "Person.h"
int main () {
        Person P1;
        P1.display();
}
```

person.h

```
#included <iostream>
class Person {
    private;
    string
     ↑Name;
      int age
    public:
       Person ();
    void display ();
}

void Meow();  // global func.
```

person.cpp

```
#include "Person.h"
#include <iostream>
using namespace std;
Person::Person () {
        cout << "Constructor";
}
void Person::display () {
        cout << Name << age;
}
void Meow () {
        cout << "Global func.";
}
```

① person.h: Class and its func. declaration / signature along with declaration / signature of global func.

② person.cpp: Defination (body) of all the func. inside class and outside it (global)

③ main.cpp: implementation of classes & objects in 'int main()'.

* "using namespace std" only required in .cpp files.. not header (.h).
* For every class, there will be seperate header files & cpps.

Gist :

If bulb is an object, use dot (.)
If bulb is pointer to object, use →
If member we're accessing a variable, then no *(_.→_)
If member we're accessing a pointer, then do *(_.→_)

this { if a variable, then this → value
if a pointer, then *(this → value).

~~* If we've an array on heap instead of pointer variable, we can just use arr[] instead of *(arr b1. arr~~

* If we've an array on heap e.g. "char * title", we can access it like "obj.title[i]". No need for * outside bracket like we need for pointer variable.

⟹ Friend func.

- C1 + 3; This calls [C1.operator + (3)] and will not need a friend func. Only '+' operator needs to be overloaded.
  Since (calling func i.e.: C1) and '+' are both in one class (i.e. complex)
  ∴ this will work without friend.
- 3 + C1;
  This will call [3.operator+(C1)] and since operator '+' isn't overloaded in calling func's (i.e : '3') class (i.e int), ∴ we'll need friend func.

~~⟹ This : 'this' is basically 'a pointer to object'.~~

```
~~class Bulb {
    private:
        int v1;
        int * v2;
    public:
        void State () {...}
        void GetV1 () {...}
        Void Ge
}
int main () {~~
```

~~bulb.v1~~ ~~bulb.state~~

[This] : this is basically 'a pointer to object'. Only used inside class

'dot' if we're object
'→' if we're pointer to object

```
class Bulb {
public:
    int value;
    int * ptr;
    void state () {
    int GetPtr () {
        return * ptr; }
}
```

```
int main () {
    Bulb b1;
    Bulb * b2;
}
```

this.value } always wrong since 'this' is
this.State() } a pointer & can't be used with a dot w/out dereferencing.

b1.value;	b1 : object	value : variable
b1.state();	b1 : object	state() : simple func.
b2 → value;	b2 : Pointer to object	value : variable
b2 → state();	b2 : Pointer to object	state() : simple func.
this → value;	value : variable	
this → state();	state() : simple func.	
*(this → ptr);	ptr : pointer variable OR pointer returning func.	
this → ptr; for accessing ptr to do this → ptr = &x		
*(b1.ptr); // b1.GetPtr(); b1 : object	ptr : pointer variable or pointer func.	
b1.ptr; for accessing ptr to do b1.ptr = &x		
*(b2 → ptr);	b2 : pointer to object	ptr : pointer variable or pointer func.

(*b2).value
(*this).value

Occurs with Inheritance & pointer.

~~[virtual func.]~~

POLYMORPHISM↑: Different objects can respond to same func. in different ways. [called "Func. Overloading"]

e.g: when we've named 2 func. of same name ↑ in base and derived class; compiler will always call the func. in base class until we use polymorphism by "virtual" keyword.

virtual ←

```
class Animal {
public:
    virtual void speak () {
        cout << " Animall speaks";
    }
};

class Dog : public Animal {
public:
    void speak() override {
        cout << " Dog barks";
    }
};
```

```
int main () {
Animal * a;
Dog d;
a = & d;
a -> speak();
}
```

Inheritance & pointers

OR

Animal *a = new Dog();

a -> Dog::speak() will work w/out virtual

Even though we assigned the address of 'd to a' here ←, C++ still would run the "speak()" func. of Base class i.e: Animal.

But since we've added "virtual" keyword ; ∴ now by doing "a -> speak()" ; the speak() func. of derived class (i.e. Dog) will be called. This enables POLYMORPHISM.

⇒ Virtual Destructors: If a pointer of Base class type is pointing a memory on heap of derived class type Animal * a = new Dog();
Then for both destructors (Animal & Dog class) to be called at the same time, we need virtual destructors with base class's destructor.

→ Code on Next Page ←

```
class Animal {
public:
    Animal () {
        cout<< "Animal created";
    }
    virtual ~Animal () {
        cout<< " Animal destroyed";
    }
class Dog : public Animal {
public :
    Dog () {
        cout<< " dog created";
    }
    ~Dog() {
        cout << "dog destroyed";
    }
```

```
int main () {
    Animal *a = new Dog();
    delete a;
}
```

calls both the destructor (of dog & animal). Otherwise only Animal destructor wld be called if no "virtual destructor" causing memory leak.

Output:

```
Animal  created
Dog     created
Dog     destroyed
Animal  destroyed.
```

⟷

⟹ Protect Members: Variables won't be accessed outside class (like private) but they'll be accessible inside derived class.

```
class Base {
protected :
    int x = 10;
};
class Derived : public Base {
public :
    void display () {
        cout << x;
    }
};
```

```
int main () {
    Derived d;
    d.display(); // output: 10.
}
```

"cout<< d.x" is error since x is not accessible outside class.

NOTE: Using "virtual" keyword is "Runtime" polymorphism

→ If we've not using 'virtual' keyword & need to call the func. of derived class that has same name as the func. in base class: →Not polymorphism

```
Dog d;
d.speak();
```

⟹ Polymorphism & Array:

```
class Animal {
public:
    virtual void Sound() {}
    virtual ~Animal() {}
};
```

```
class Cat : public Animal {
public :
    void Sound() override {}
};
```

```
class Cow : public Animal {
public
    void Sound() override {}
};
```

```
int main() {
    Animal * animals[2];
    animals [0] = new Cat();
    animals [1] = new Cow();
    for (int i=0; i< 2 ; i++) {
        animals [i] -> sound();
    }
    for (int i=0; i<2; i++) {
        delete animals [i];
    }
```

→ will output/run 'Cat' 'sound' func. first and then 'Cow' 'sound' func.

→ Not "delete [] animals" since 'animals[2] array is on Stack not on heap

Animal a = new Animals [2]